



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие №3

Студенты группы *ИКБО-20-23 Кузнецов Лев Андреевич*

(подпись)

Ассистент *Брайловский А.В.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

СОДЕРЖАНИЕ

ПОСТАНОВКА ЗАДАЧИ	3
ХОД РАБОТЫ	5
ВЫВОД.....	11

ПОСТАНОВКА ЗАДАЧИ

Цель: формирование глубокого понимания и практического применения инструментов для реализации сложной бизнес-логики непосредственно на уровне базы данных.

Постановка задачи:

Задание 1: использование оператора CASE

1. Составить запрос, использующий поисковое выражение CASE для категоризации данных по какому-либо числовому признаку из вашей БД (например, цена, количество, возраст). Запрос должен содержать не менее трех условий WHEN и ветку ELSE.

2. Составить запрос, в котором оператор CASE используется внутри агрегатной функции (например, SUM или COUNT) для выполнения условной агрегации.

Задание 2: использование подзапросов.

Составить и выполнить три запроса, демонстрирующих разные типы подзапросов.

1. Скалярный подзапрос: найти все записи в таблице, у которых значение в некотором числовом столбце превышает среднее (или максимальное/минимальное) значение по этому столбцу.

2. Многострочный подзапрос с IN: вывести информацию из одной таблицы на основе идентификаторов, полученных из связанной таблицы по определенному критерию (в данном случае, обязательно по дате).

3. Коррелированный подзапрос с EXISTS: найти все записи из родительской таблицы, для которых существует хотя бы одна связанная запись в дочерней таблице, удовлетворяющая текстовому условию.

4. Альтернативное решение с JOIN: решите задачу из пункта выше (2.3, Коррелированный подзапрос с EXISTS), но на этот раз с использованием оператора соединения JOIN.

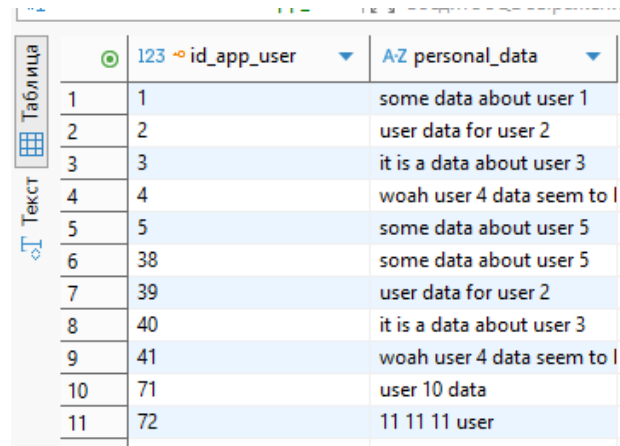
Задание 3: использование обобщенных табличных выражений (CTE).

1. Стандартное CTE: переписать запрос из Задания 2.3 (с коррелированным подзапросом) с использованием обобщенного табличного выражения (CTE).

2. Рекурсивное CTE: используя имеющуюся в вашей схеме данных таблицу с иерархической структурой (например, pharmacists), написать рекурсивный запрос с помощью WITH RECURSIVE для вывода всей иерархии с указанием уровня вложенности.

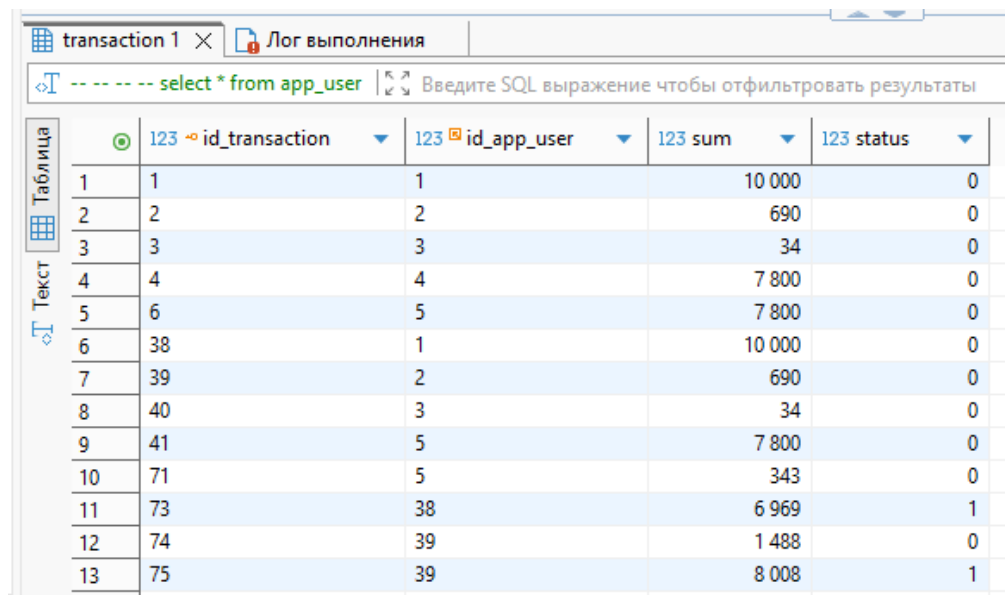
ХОД РАБОТЫ

Используемые таблицы



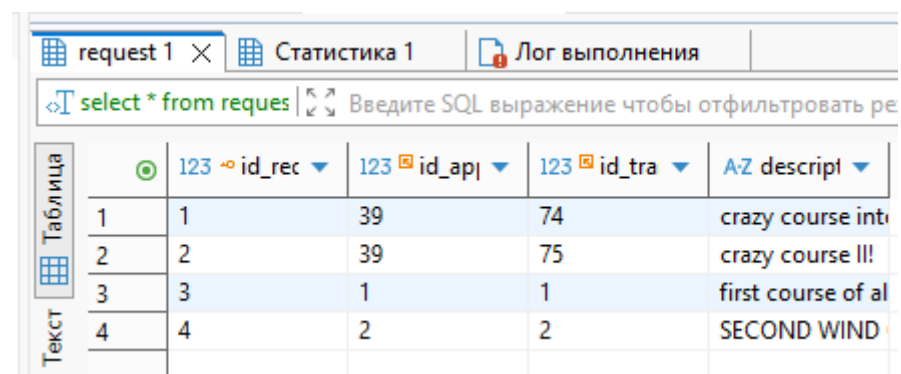
	123 id_app_user	AZ personal_data
1	1	some data about user 1
2	2	user data for user 2
3	3	it is a data about user 3
4	4	woah user 4 data seem to l
5	5	some data about user 5
6	38	some data about user 5
7	39	user data for user 2
8	40	it is a data about user 3
9	41	woah user 4 data seem to l
10	71	user 10 data
11	72	11 11 11 user

Рисунок 1 – Таблицы app_user



	123 id_transaction	123 id_app_user	123 sum	123 status
1	1	1	10 000	0
2	2	2	690	0
3	3	3	34	0
4	4	4	7 800	0
5	6	5	7 800	0
6	38	1	10 000	0
7	39	2	690	0
8	40	3	34	0
9	41	5	7 800	0
10	71	5	343	0
11	73	38	6 969	1
12	74	39	1 488	0
13	75	39	8 008	1

Рисунок 2 – Таблицы app_user



	123 id_rec	123 id_apl	123 id_tra	AZ descript
1	1	39	74	crazy course int
2	2	39	75	crazy course III
3	3	1	1	first course of al
4	4	2	2	SECOND WIND

Рисунок 3 – Таблицы request

	recursion_id	name	parent_id
1	1	Электроника	[NULL]
2	2	Бытовая техника	[NULL]
3	3	Смартфоны	1
4	4	Ноутбуки	1
5	5	Холодильники	2
6	6	Аксессуары для смартфонов	3

Рисунок 4 – Таблицы recursive_table

1. Реализация условной логики – оператор CASE

```

-- 2 по какому из запросов
-- CTE 2 вариации
-- создать рекурсии при помощи менеджера у одного из сотрудников
-- DON'T READ COMMENTS
--with mind as(
-- select *
-- from request r
--)
select
  t.sum,
  case
    when t.sum < 1000 then 'cheap'
    when t.sum < 10000 then 'medium'
    else 'costy'
  end as price_category
from
  transaction as t
--join
-- mind on true

```

	sum	price_category
1	10 000	costy
2	690	cheap
3	34	cheap
4	7 800	medium
5	7 800	medium
6	10 000	costy
7	690	cheap
8	34	cheap
9	7 800	medium
10	343	cheap
11	6 969	medium
12	1 488	medium
13	8 008	medium

Рисунок 1.1 – Применение оператора CASE

```

--with mind as(
-- select *
-- from request r
--)
select
  t.sum,
  count(
    case
      when t.sum < 1000 then 'cheap'
      else null
    end
  ) as price_category
from
  transaction as t
group by
  t.sum
--join
-- mind on true
-- case: if else

```

	sum	price_category
1	1 488	0
2	10 000	0
3	8 008	0
4	34	2
5	6 969	0
6	690	2
7	7 800	0
8	343	1

Рисунок 1.2 – Результат работы с условной агрегацией

2. Использование подзапросов

```

-- выбраны все транзакции, где t.sum > среднего значения
select
  t.id_transaction,
  t.sum
from
  transaction as t
where
  t.sum > (select avg(sum) from transaction);
--join
-- mind on true
-- case: if else

```

	id_transaction	sum
1	1	10 000
2	4	7 800
3	6	7 800
4	38	10 000
5	41	7 800
6	73	6 969
7	75	8 008

Рисунок 2.1 – Применение скалярного подзапроса

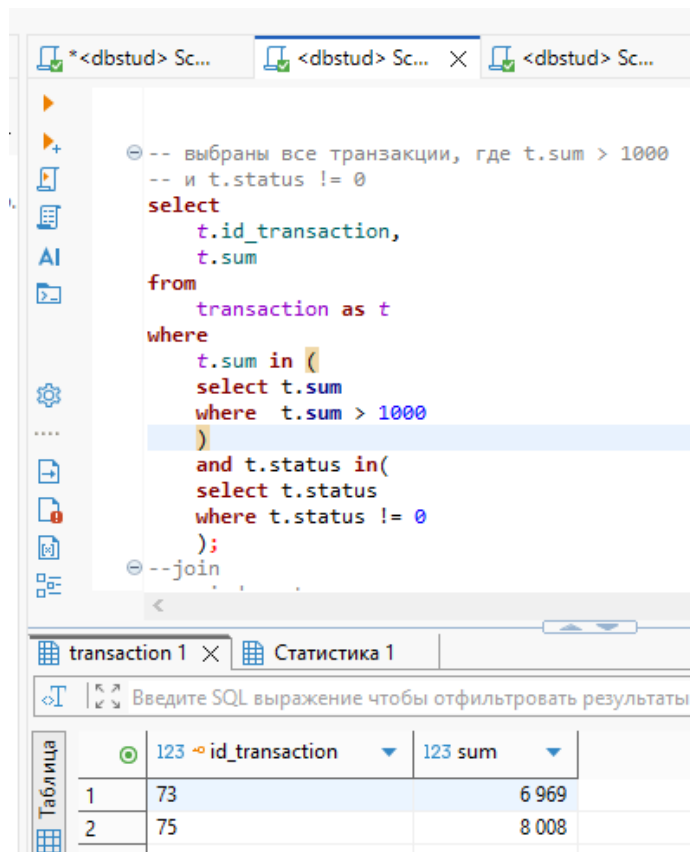


Рисунок 2.2 – Применение подзапроса IN

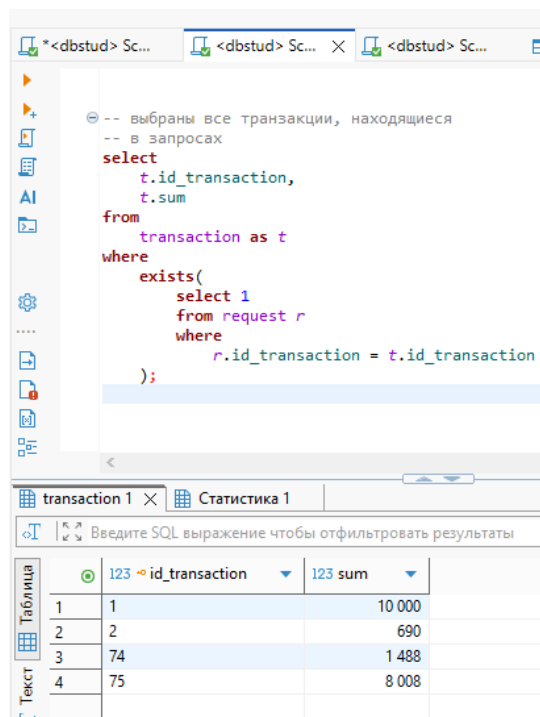


Рисунок 2.3 – Применение коррелированного подзапроса EXISTS

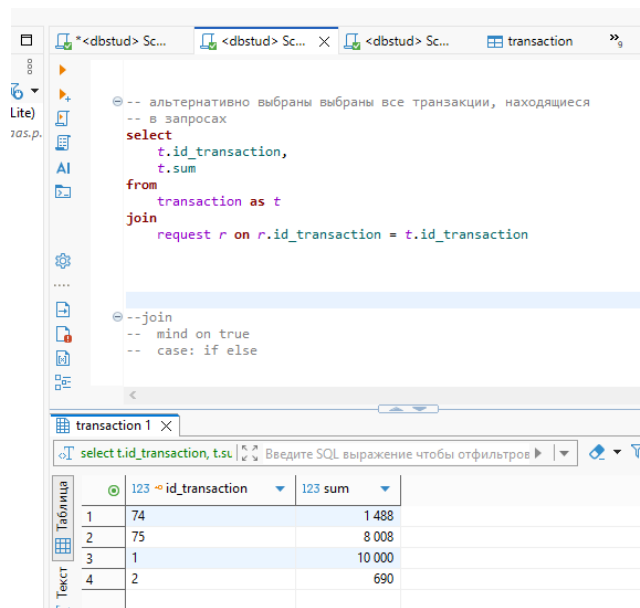


Рисунок 2.4 – Альтернативное решение поставленной задачи без применения EXISTS

3. Использование обобщённых табличных выражений (СТЕ)

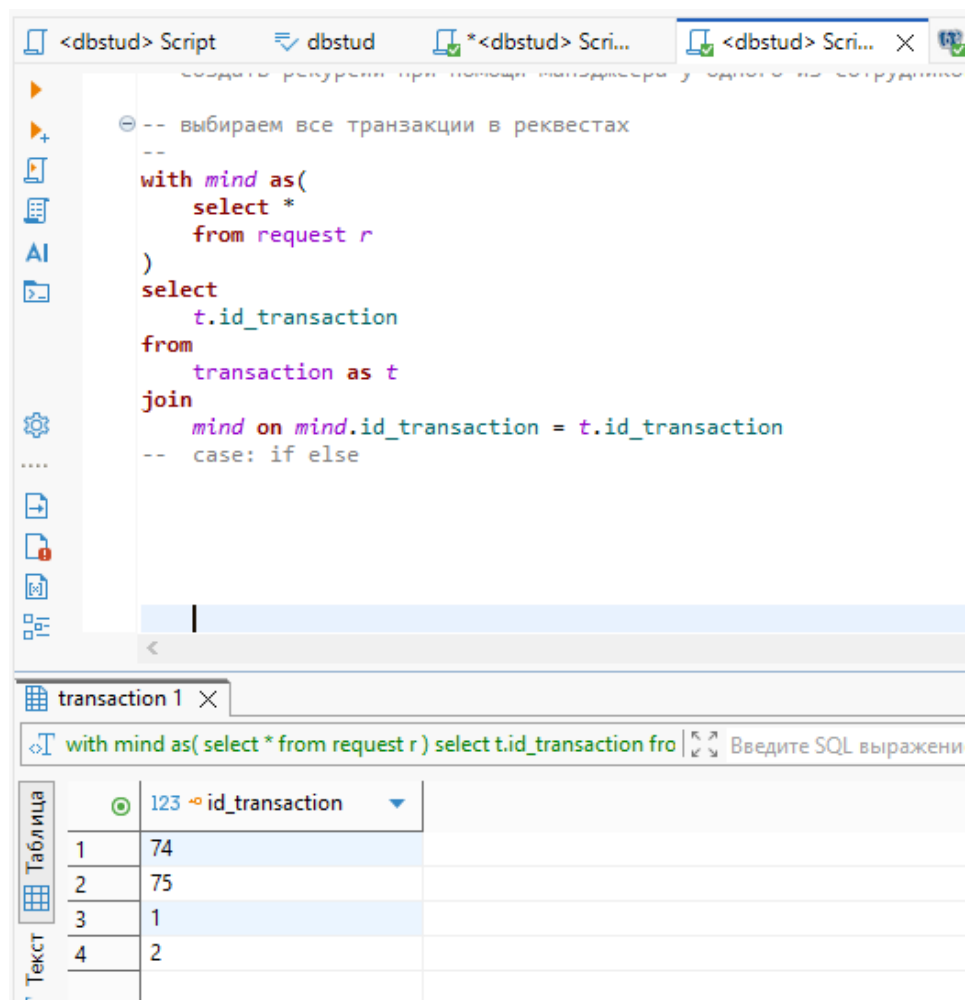


Рисунок 3.1 – Стандартное использование СТЕ

<pre> -- рекурсивно выбираем все таблицы -- по их родителю with recursive recursion as(select recursion_id, name, parent_id, 0 as level from recursive table where parent_id is null union all select r_t.recursion_id, r_t.name, r_t.parent_id, r.level + 1 as level from recursive table as r_t join recursion as r on r.recursion_id = r_t.parent_id) select * from recursion; </pre>																																															
Результат 1 Введите SQL выражение чтобы отфильтровать результаты <table> <tr> <th></th><th>123 recursion_id</th><th>AZ name</th><th>123 parent_id</th><th>123 level</th><th></th></tr> <tr> <td>1</td><td>1</td><td>Электроника</td><td>[NULL]</td><td>0</td><td></td></tr> <tr> <td>2</td><td>2</td><td>Бытовая техника</td><td>[NULL]</td><td>0</td><td></td></tr> <tr> <td>3</td><td>3</td><td>Смартфоны</td><td>1</td><td>1</td><td></td></tr> <tr> <td>4</td><td>4</td><td>Ноутбуки</td><td>1</td><td>1</td><td></td></tr> <tr> <td>5</td><td>5</td><td>Холодильники</td><td>2</td><td>1</td><td></td></tr> <tr> <td>6</td><td>6</td><td>Аксессуары для с</td><td>3</td><td>2</td><td></td></tr> </table>							123 recursion_id	AZ name	123 parent_id	123 level		1	1	Электроника	[NULL]	0		2	2	Бытовая техника	[NULL]	0		3	3	Смартфоны	1	1		4	4	Ноутбуки	1	1		5	5	Холодильники	2	1		6	6	Аксессуары для с	3	2	
	123 recursion_id	AZ name	123 parent_id	123 level																																											
1	1	Электроника	[NULL]	0																																											
2	2	Бытовая техника	[NULL]	0																																											
3	3	Смартфоны	1	1																																											
4	4	Ноутбуки	1	1																																											
5	5	Холодильники	2	1																																											
6	6	Аксессуары для с	3	2																																											

Рисунок 3.2 – Рекурсивное использование CTE

ВЫВОД

В ходе выполнения работы были получены базовые навыки для работы с case, обобщёнными табличными выражениями и корреляционными подзапросами. Полученные знания были закреплены путём решения поставленных задач: составление рекурсии, альтернативные способы получения одинакового результата и создание на ходу таблиц.